Day 24 — handling imbalanced classes: complete guide

MD Mutasim Billah · 52-week Data Science to ML/AI roadmap · Week 4, Day 4 — definitions, real-world examples, implementations, line-by-line code, and interview prep — paired with day24_imbalanced_classes.py

Same complete format as Day 21-23: for every topic, a definition, a real-world example, the real implementation, and a line-by-line walk-through of what each part of the code does and why. A technical and theoretical interview quiz closes out the guide.

1. Why accuracy is misleading on imbalanced data

Definition — Class imbalance: when one class makes up a much larger share of the dataset than another. Titanic's Survived column is mildly imbalanced: roughly 62% died, 38% survived (varies slightly by the exact sample).

Real-world example: A hospital screening for a rare disease that affects 2% of patients could build a model that always predicts "healthy" and score 98% accuracy — while missing every single actual case. The number looks excellent and is completely useless. Titanic's imbalance is milder, but the same distortion applies: a model that always predicts "died" scores roughly 62-78% accuracy (depending on the exact split) while catching zero real survivors.

```
print(y.value_counts(normalize=True))
# died        0.62 (or similar)
# survived    0.38

print(f"Always-died baseline accuracy: {(y == 0).mean():.1%}")
```

→ `value_counts(normalize=True)` — returns proportions instead of raw counts, making the imbalance immediately visible as percentages rather than needing to compute the ratio by hand

→ printing this always-died baseline number before training anything is a habit worth keeping on any classification problem — it's the actual floor any real model needs to beat, not 50%

2. class_weight='balanced': reweighting the loss

Definition — class_weight='balanced': a parameter accepted by most scikit-learn classifiers that automatically weights each class inversely proportional to how often it appears, so mistakes on the rarer class cost more during training itself.

Real-world example: It's the training-time equivalent of a teacher deciding that missing a rare, important exam question costs more points than missing a common, easy one — the grading rubric itself is changed, not just how the final grades get curved afterward.

```
weighted_model = RandomForestClassifier(
    n_estimators=200, random_state=42, class_weight="balanced"
)
```

→ `class_weight="balanced"` — automatically computes weights as n_samples / (n_classes × class_count), so the minority class gets a larger weight without needing to calculate or hardcode the exact ratio by hand

→ this changes what the model optimizes for during `.fit()`, unlike Day 18's threshold adjustment, which only changes how an already-trained model's output gets converted to a label afterward

3. Oversampling vs. undersampling

Definition — Oversampling: adds more examples of the minority class to the training data until the classes are closer to balanced.

Definition — Undersampling: removes examples of the majority class instead, until the classes are closer to balanced.

Real-world example: With only 320 training rows and roughly 38% survivors, undersampling would mean throwing away real majority-class data on an already small dataset — a genuine tradeoff, not a free option. Oversampling instead adds to the minority class rather than subtracting from the majority, which is generally preferred when the dataset is already small.

4. SMOTE: synthetic, not duplicated, minority examples

Definition — SMOTE (Synthetic Minority Oversampling Technique): generates new minority-class examples by interpolating between real minority examples and their nearest neighbors in feature space, rather than simply duplicating existing rows.

Real-world example: Naive oversampling (just copying survivor rows) is like photocopying a handful of exam answers and adding the copies to the study set — the model can memorize those exact copies without actually learning the underlying pattern. SMOTE instead creates plausible new points that sit between real survivors in feature space — a passenger the model never literally saw, but who resembles genuine survivors closely enough to be a reasonable synthetic example.

```
from imblearn.over_sampling import SMOTE

smote = SMOTE(random_state=42)
X_resampled, y_resampled = smote.fit_resample(X_train_transformed, y_train)
```

→ `fit_resample()` — unlike every transformer used so far, this returns both a new X and a new y, because resampling changes the number of rows, not just the columns

→ SMOTE operates on already-numeric data (it needs to compute distances between points to interpolate), which is exactly why it has to run after preprocessing, not before

5. Why resampling must happen only on training folds

Definition — imblearn.pipeline.Pipeline: a drop-in alternative to scikit-learn's Pipeline that additionally supports sampling steps (like SMOTE) between preprocessing and the classifier — something scikit-learn's own Pipeline cannot do, because sampling changes the number of rows, which standard transformers are never allowed to do.

Real-world example: If SMOTE ran before the train/test split, some synthetic points could be interpolated using information derived from rows that end up in the test set, and the test set itself could end up partly synthetic — evaluating a model on data that was never real is meaningless. Just like Day 21-22's leakage fixes, the rule is the same: anything learned or generated from data must happen using training data only.

```
from imblearn.pipeline import Pipeline as ImbPipeline

smote_pipeline = ImbPipeline(steps=[
    ("engineer", engineer),
    ("preprocessor", preprocessor),
    ("smote", SMOTE(random_state=42)),
    ("classifier", RandomForestClassifier(n_estimators=200, random_state=42)),
])
```

→ `from imblearn.pipeline import Pipeline as ImbPipeline` — imported under a different name specifically to avoid confusion with scikit-learn's own Pipeline used elsewhere in this same file

→ the `"smote"` step sits between preprocessing and the classifier — it needs already-numeric data (from the preprocessor) to compute distances, but needs to run before the classifier ever sees the data

→ during `.fit()`, the SMOTE step runs and adds synthetic rows before the classifier trains; during `.predict()`, the SMOTE step is automatically skipped entirely, since there's never a reason to synthesize new rows for data you're trying to score

6. Comparing baseline vs. class_weight vs. SMOTE

Judging these three approaches requires looking specifically at the survived class's precision, recall, and F1 — not overall accuracy, since accuracy is exactly the metric that rewards ignoring the minority class in the first place.

```
report = classification_report(
    y_test, preds, target_names=["died", "survived"], output_dict=True
)
survived_recall = report["survived"]["recall"]
survived_precision = report["survived"]["precision"]
```

→ `output_dict=True` — returns the report as a nested dictionary instead of a printable string, making it possible to pull out just the survived class's numbers programmatically for the comparison table

→ there's no universally correct answer between class_weight and SMOTE — the script compares both explicitly on the same held-out test set rather than assuming one is always better, and in practice the winner can vary by dataset

Interview prep quiz — technical

Questions about class_weight, SMOTE, and imblearn.Pipeline syntax.

Q: What does class_weight='balanced' actually compute, and when does it take effect?

A: It automatically computes a weight per class inversely proportional to that class's frequency in the training data, and applies those weights during .fit() so mistakes on the minority class are penalized more heavily while the model is being trained.

Q: Why does SMOTE's fit_resample() return both X and y, unlike a normal transformer's transform()?

A: Because resampling changes the number of rows in the dataset — new synthetic rows are added — so both the feature matrix and the target array need to grow together to stay aligned, unlike a transformer that only changes column values without changing row count.

Q: Why can't SMOTE be used inside a regular scikit-learn Pipeline?

A: Scikit-learn's Pipeline requires every step to preserve the number of rows between input and output. SMOTE adds new rows, which violates that assumption — imblearn.pipeline.Pipeline is a separate implementation built specifically to support sampling steps that change row counts.

Q: Why must the SMOTE step come after the preprocessor step in the pipeline, not before?

A: SMOTE needs to compute distances between points to interpolate new synthetic examples, which requires numeric data. Raw categorical text columns like Sex or Embarked need to be encoded first, which is exactly what the preprocessor step does.

Q: What happens to the SMOTE step in an imblearn Pipeline when .predict() is called, versus .fit()?

A: During .fit(), SMOTE generates synthetic minority rows before the classifier trains. During .predict(), the SMOTE step is automatically skipped — there's no reason to synthesize new rows for data you're trying to generate predictions for.

Interview prep quiz — theoretical / conceptual

Questions about why imbalance matters and how to reason about fixing it.

Q: Why is accuracy specifically the wrong metric to use when deciding whether class_weight or SMOTE actually helped?

A: Accuracy is the exact metric that made the imbalance problem invisible in the first place — a model that ignores the minority class entirely can still score high accuracy. Judging the fix using the same metric that hid the problem risks concluding a fix 'didn't help' when it actually improved what matters most: catching minority-class cases.

Q: Why might undersampling be a worse choice than oversampling on a small dataset like this one, even though both aim to balance classes?

A: Undersampling throws away real majority-class rows to reach balance, which on an already small dataset (320 training rows here) means training on meaningfully less real data overall. Oversampling adds to the minority class instead of subtracting from the majority, preserving all the real data that already exists.

Q: Why is generating synthetic data with SMOTE before the train/test split considered a leakage risk, even though the synthetic points aren't real passengers?

A: Synthetic points are interpolated from real minority examples — if those real examples end up split across both train and test, or if synthetic points are generated using information from what becomes the test set, the test set is no longer a clean, untouched measure of generalization. Evaluating on partially synthetic data would also be meaningless, since synthetic points were never real outcomes to predict.

Q: In a business setting, how would you decide whether to prioritize catching more true positives (higher recall) versus avoiding false alarms (higher precision) when choosing between class_weight, SMOTE, or neither?

A: That decision depends on the relative cost of each type of mistake in the specific context — missing a real survivor versus a fraud case might be far more costly than a false alarm, while in another setting the reverse might be true. The right approach is chosen based on that cost tradeoff, not by defaulting to whichever technique scores highest on a single blended metric.